

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Modal

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

Assessment Tool Weightage: 30%

QUESTIONS:

Total Marks:25

Q1. Transfer Learning

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

Q2. DenseNet and PixelNet

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

Q3. Bidirectional RNN and Encoder–Decoder

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

Q4. BPTT and LSTM

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

Q5. Integrated Deep Learning Model

Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture. Justify the role of each component and explain how the complete system generates meaningful captions.

Code of Conduct:

I B.Jahnavi / 192325043 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: jahnavi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q1. Transfer Learning – Plant Disease Detection

1. Problem Understanding

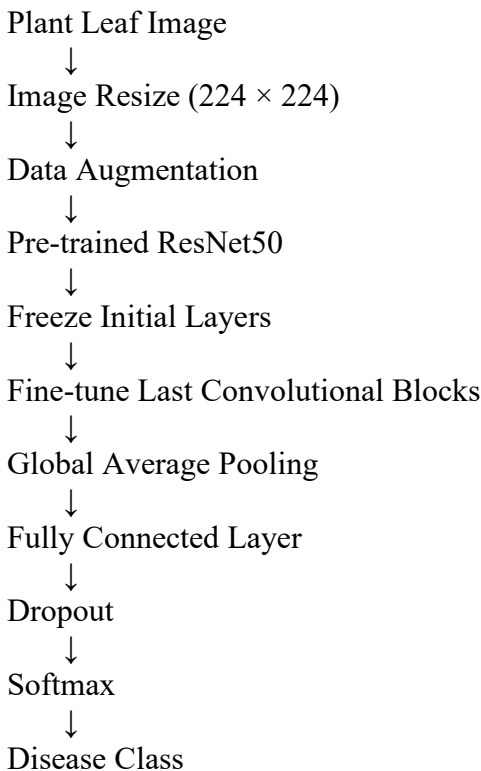
The objective is to design a plant disease detection system using a **pre-trained CNN**. The system receives a leaf image and predicts the corresponding disease class. Transfer learning is suitable because a pre-trained CNN has already learned useful low-level and high-level visual features from a large image dataset.

The major challenges are:

- Different lighting and background conditions.
- Variations in leaf size, orientation and shape.
- Similar visual symptoms between diseases.
- Limited availability of labelled agricultural images.
- Class imbalance between common and rare diseases.

2. Proposed Architecture

A suitable model is ResNet50 pre-trained on ImageNet.



Layer Selection

Initially, the convolutional layers are frozen because they contain general features such as:

- edges
- textures
- shapes
- colour patterns

The final layers are fine-tuned because they need to learn plant-specific disease features.

For example:

ResNet50

— Initial convolutional layers	→ Frozen
— Middle layers	→ Frozen initially
— Final residual blocks	→ Fine-tuned
— Classification head	→ Newly trained

After the classification head has converged, some deeper layers can be unfrozen and trained with a small learning rate.

3. Data Augmentation

The following transformations improve generalization:

- Random rotation
- Horizontal/vertical flipping where biologically appropriate
- Random zoom
- Random crop
- Brightness variation
- Contrast variation
- Small translations

For example:

```
augmentation = [RandomFlip("horizontal"),RandomRotation(0.2),RandomZoom(0.2),  
RandomContrast(0.2)]
```

4. Training Algorithm

1. Load the plant disease dataset.
2. Divide it into training, validation and test sets.
3. Resize images to 224×224 .
4. Apply augmentation to training images.
5. Load pre-trained ResNet50.
6. Freeze the initial convolutional layers.
7. Replace the original classification layer.
8. Train the new classification head.
9. Unfreeze the final convolutional blocks.
10. Fine-tune using a small learning rate.
11. Evaluate using accuracy, precision, recall and F1-score.

Complexity

For an input image of fixed size, inference requires approximately the computational cost of the selected CNN. Transfer learning reduces training cost because most parameters are initially frozen.

5. Performance

Performance can be improved using:

- learning-rate scheduling
- early stopping
- dropout
- class-weighted loss
- balanced sampling
- batch normalization
- fine-tuning only deeper layers

Accuracy alone should not be used for an imbalanced dataset. **Precision, recall and F1-score** should also be considered.

6. Test Cases

Test Case	Input	Expected Output
TC1	Healthy leaf	Healthy
TC2	Clearly diseased leaf	Correct disease
TC3	Rotated leaf	Same disease
TC4	Low-light image	Correct/robust prediction
TC5	Unseen background	Disease classification
TC6	Poor-quality image	Low-confidence/appropriate prediction

Conclusion

Transfer learning provides an efficient solution because general visual features from the pre-trained CNN can be reused while the deeper layers are adapted to plant disease characteristics.

Q2. DenseNet + PixelNet for Semantic Segmentation

1. Problem Understanding

The objective is to segment autonomous-driving images into pixel-level classes such as:

- road
- vehicle
- pedestrian
- building
- vegetation
- sky
- traffic signs

Unlike image classification, semantic segmentation must assign a class to every pixel. The assessment specifically requires integration of DenseNet and PixelNet concepts.

2. Proposed Architecture

Input Road Image



Pre-processing



DenseNet Encoder



Dense Feature Extraction



Multi-scale Feature Fusion



PixelNet-style Pixel-wise Prediction



Upsampling



1×1 Convolution



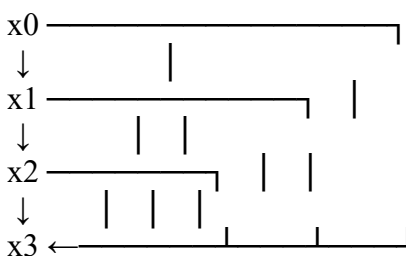
Softmax



Pixel-level Segmentation Map

Dense Connectivity

In DenseNet, each layer receives feature maps from preceding layers:



Mathematically:

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

where $[]$ represents concatenation.

This improves feature reuse and gradient propagation.

3. Pixel-wise Prediction

PixelNet-style processing produces predictions for individual pixels.

For every pixel p :

$$P(y_p=c | X) = \text{Softmax}(f(X_p))$$

where:

- X_p = feature representation of pixel p
- c = semantic class
- f = prediction network

The final output is:

Input:

[Road Image]

Output:

[Road][Road][Car][Car]
[Road][Person][Car][Road]
[Road][Road][Road][Building]

4. Training Algorithm

A suitable loss function is pixel-wise cross entropy:

$$L = -p \sum_c \sum_{y_p} y_p \log(\hat{y}_p^c)$$

For highly imbalanced classes, weighted cross entropy can be used.

Training steps:

1. Load labelled driving images.
2. Resize and normalize images.
3. Pass images through DenseNet encoder.
4. Extract multi-level features.
5. Fuse features.
6. Upsample feature maps.
7. Generate pixel-level predictions.
8. Calculate segmentation loss.
9. Backpropagate gradients.
10. Update network parameters.

5. Why Accuracy Improves

DenseNet helps because:

- Features are reused.
- Gradient flow is improved.
- Earlier visual information is preserved.
- Deep representations contain richer context.

Pixel-wise prediction helps because:

- Every pixel receives a classification.
- Object boundaries can be preserved.
- Small objects such as pedestrians and signs can be detected.
- Spatial information is retained.

6. Performance Metrics

Important segmentation metrics include:

$$\text{IoU} = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$$

and

$$\text{mIoU} = 1/C \sum_{c=1}^C \text{IoU}_c$$

where C is the number of classes.

Dice coefficient can also be used:

$$\text{Dice} = 2\text{TP} / (2\text{TP} + \text{FP} + \text{FN})$$

7. Test Cases

Test Case	Situation	Expected Result
TC1	Normal road	Accurate road segmentation
TC2	Multiple vehicles	Vehicles separated from road
TC3	Pedestrian	Pedestrian pixels detected
TC4	Night image	Robust segmentation
TC5	Small traffic sign	Sign detected
TC6	Occluded vehicle	Reasonable partial segmentation

Conclusion

The combination of DenseNet feature reuse and pixel-wise prediction creates a strong architecture for autonomous-driving semantic segmentation.

Q3. Bidirectional RNN + Encoder–Decoder for Machine Translation

1. Problem Understanding

The objective is to create a multilingual sequence-to-sequence translation system. The model must convert a source-language sentence into a target-language sentence.

For example:

English:

"I love machine learning"

↓ Encoder

Context representation

↓ Decoder

French:

"J'aime l'apprentissage automatique"

The question specifically requires a Bidirectional RNN encoder–decoder and explanation of how bidirectional context improves translation.

2. Architecture

Source Sentence



Tokenization



Word Embeddings



Bidirectional RNN Encoder



Context Representation



Attention / Context Vector



RNN Decoder



Target Word Prediction



Translated Sentence

Bidirectional Encoder

A BiRNN processes the sentence in two directions.

Forward:

$$h_t = f(x_t, h_{t-1})$$

Backward:

$$h_t = f(x_t, h_{t+1})$$

The final representation is:

$ht = [ht; ht]$

Thus, each word has information from both its past and future context.

3. Example

Consider: "I went to the bank to deposit money."

The meaning of "bank" becomes clearer from surrounding words such as "deposit" and "money".

A unidirectional encoder primarily uses previous context, whereas the BiRNN representation incorporates information from both directions.

4. Decoder

The decoder generates one word at a time:

<START>
↓
Word 1
↓
Word 2
↓
Word 3
↓
...
↓
<END>

The probability of the next word can be represented as:

$P(y_t \mid y_1, \dots, y_{t-1}, X)$

Training uses teacher forcing, where the correct previous target word is provided to the decoder.

5. Training Algorithm

1. Tokenize source and target sentences.
2. Convert tokens into embeddings.
3. Pass source sequence through BiRNN.
4. Generate contextual representations.
5. Initialize decoder.
6. Feed target sequence using teacher forcing.
7. Predict next target token.
8. Calculate cross-entropy loss.
9. Perform backpropagation through time.
10. Update parameters.

6. Performance

Performance can be measured using:

- BLEU score
- validation loss
- translation accuracy
- inference time

Beam search can be used during inference to generate better translations.

7. Test Cases

Test Case	Input	Expected
TC1	Short sentence	Correct translation
TC2	Long sentence	Context preserved
TC3	Unknown word	Reasonable handling
TC4	Ambiguous word	Context-dependent translation
TC5	Different language pair	Correct target language

Conclusion

The BiRNN encoder improves translation because it captures both previous and future context, providing the decoder with a richer representation of the source sentence.

Q4. BPTT + LSTM for Stock Market Trend Prediction

1. Problem Understanding

The objective is to predict future stock-market trends from historical time-series data. The input can contain:

- opening price
- closing price
- high price
- low price
- trading volume
- technical indicators

The model predicts either a future price or a trend such as:

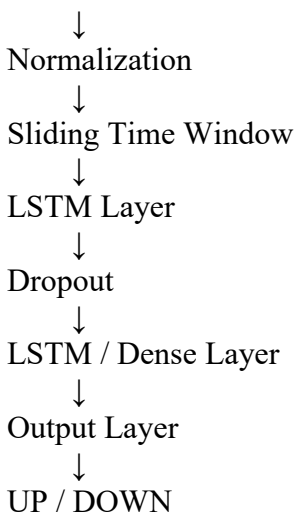
UP

DOWN

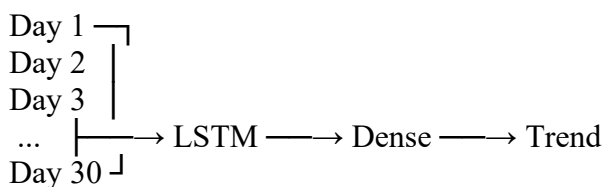
The assessment specifically asks for LSTM and explanation of Backpropagation Through Time (BPTT).

2. Proposed Architecture

Historical Stock Data



For example, use the previous 30 trading days to predict the next day's trend.



3. LSTM Structure

The LSTM uses three important gates.

Forget gate : $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

Input gate : $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$

Output gate : $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

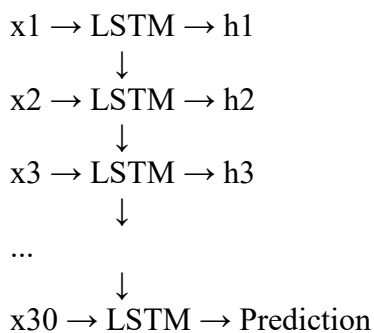
Cell state: $C_t = f_t C_{t-1} + i_t \tilde{C}_t$

Hidden state: $h_t = o_t \tanh(C_t)$

These gates allow LSTM to control what information is retained or discarded.

4. BPTT

During training, the LSTM is conceptually unfolded across time:



The error at the output is propagated backwards through the sequence.

This is Backpropagation Through Time.

The model calculates gradients for parameters across the time steps and updates them using an optimizer such as Adam.

5. Why LSTM Instead of Standard RNN?

Standard RNNs can suffer from:

- vanishing gradients
- exploding gradients
- difficulty learning long-term dependencies

LSTM addresses these problems using its gated cell-state mechanism.

For stock sequences, an event many time steps earlier may influence a future trend. LSTM is therefore better suited to learning longer temporal relationships.

6. Training

Historical data



Clean missing values



Normalize features



Create sequences



Train/validation/test split



LSTM training using BPTT



Validation



Test prediction

The time-series split must preserve chronological order to prevent future information from leaking into training.

7. Performance Metrics

For trend classification:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

For price regression:

- MAE
- MSE
- RMSE

8. Test Cases

Test Case	Condition	Expected
TC1	Normal historical sequence	Trend predicted
TC2	Missing data	Preprocessing handles it
TC3	Sudden price movement	Model produces prediction
TC4	Long sequence	LSTM retains useful information
TC5	Unseen test period	Generalized prediction

Conclusion

LSTM is preferable because its memory cells and gates allow it to retain important temporal information while BPTT provides the mechanism for training the sequence model.

Q5. Integrated Deep Learning Model – Automatic Video Caption Generation

1. Problem Understanding

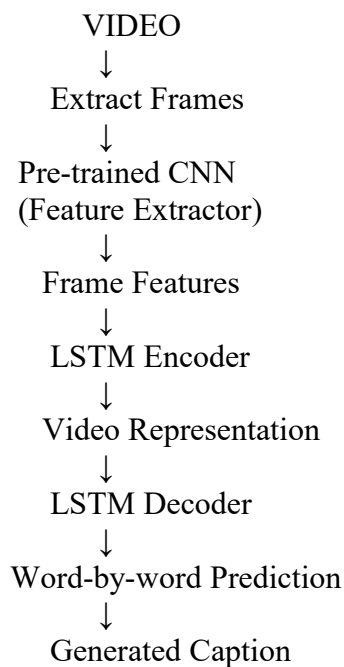
The goal is to generate a meaningful natural-language caption from a video.

For example:

Input Video
↓
Person riding a bicycle
↓
Generated Caption:
"A person is riding a bicycle."

The question requires integration of transfer learning for feature extraction with an LSTM encoder–decoder architecture.

2. Overall Architecture



3. Transfer Learning Component

A pre-trained CNN such as ResNet can extract visual features from each video frame.

Instead of training the CNN from scratch:

Video Frame
↓
Pre-trained CNN
↓
Feature Vector

The CNN can initially be frozen to reduce training cost.

Transfer learning is beneficial because the CNN already contains useful visual representations.

4. LSTM Encoder

The extracted frame features are passed sequentially to the LSTM.

$$H_t = \text{LSTM}(x_t, h_{t-1})$$

where:

- x_t = feature vector of frame t
- h_t = hidden representation at time t

The encoder learns the temporal information contained in the video.

5. LSTM Decoder

The decoder converts the learned video representation into words.

<START>
↓
A
↓
man
↓
is
↓
playing
↓
football
↓
<END>

At each time step:

$$P(y_t | y_{<t}, V)$$

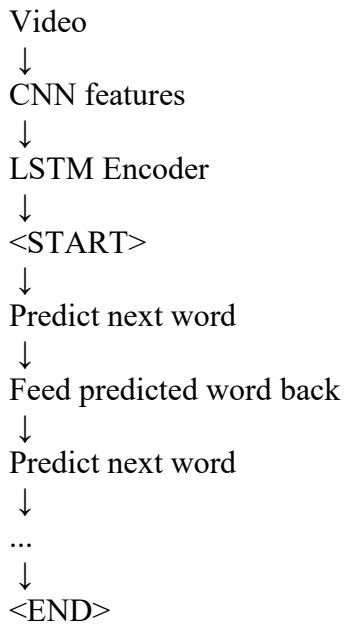
is calculated to determine the probability of the next word.

6. Training Algorithm

1. Collect videos and corresponding captions.
2. Extract representative frames.
3. Resize and normalize frames.
4. Pass frames through the pre-trained CNN.
5. Extract feature vectors.
6. Feed feature sequence to the LSTM encoder.
7. Initialize the decoder using the encoded representation.
8. Generate caption tokens.
9. Calculate cross-entropy loss.
10. Apply BPTT through the LSTM.
11. Update trainable parameters.
12. Fine-tune selected CNN layers if necessary.

7. Caption Generation

During inference:



Greedy search can choose the highest-probability word at each step. Beam search can maintain multiple candidate captions and generally provide better sequence-level results.

8. Performance

Evaluation can use:

- BLEU
- ROUGE
- METEOR
- CIDEr

Performance can be improved using:

- better frame sampling
- attention mechanisms
- beam search
- pretrained CNNs
- dropout
- data augmentation
- fine-tuning selected CNN layers

9. Test Cases

Test Case	Video	Expected Caption Behaviour
TC1	Person walking	Caption describes walking
TC2	Person playing football	Football activity identified
TC3	Multiple people	Relevant subjects identified
TC4	Long video	Important temporal information retained
TC5	Unusual scene	Reasonable generalized caption
TC6	Poor-quality frames	System remains reasonably robust

10. Complete System

The complete pipeline is:

Video→Frames→CNN→Features→LSTM Encoder→LSTM Decoder→Caption

Role of Each Component

Component	Role
Video frames	Visual input
Pre-trained CNN	Spatial feature extraction
Transfer learning	Reuse learned visual knowledge
LSTM Encoder	Learn temporal relationships
LSTM Decoder	Generate language sequence
Softmax	Predict next word
Beam search	Improve caption selection

Conclusion

The integrated model combines the strengths of two deep-learning approaches: transfer learning efficiently extracts visual information, while the LSTM encoder–decoder learns temporal relationships and generates captions sequentially.